

FINAL CLASS: REAL-WORLD PHP OOP WITH MYSQL

Building a Secure Student Management Dashboard (CRUD + Frontend)

Course Position: Final class of PHP OOP

Level: Beginner → Intermediate

Outcome: Students finish with real-world confidence and readiness for frameworks (Laravel, Symfony)

1 CLASS GOALS (WHY THIS CLASS MATTERS)

By the end of this final class, students will:

-  **Advanced but still framework-free** architecture (Front Controller + Router)
 -  **PSR-4 style autoloading** (simple Composer option + fallback)
 -  **.env support** (real-world config approach) + safe fallback
 -  **Validation with per-field errors** (not just one message)
 -  **Flash messaging + old input** (like Laravel)
 -  **CSRF protection**
 -  **Method spoofing** (PUT/DELETE via POST) — real-world pattern
 -  **Repository-like model** + cleaner DB wrapper methods
 -  **Search + pagination**
 -  **Soft delete** option (bonus) + instructions
 -  **Security notes** (XSS, SQLi, CSRF, error handling)
 -  **Exercises + challenge upgrades**
-

Project: Student Management Dashboard (Admin CRUD)

This final project is designed to feel like a **mini Laravel** without using any framework—so students understand what frameworks do behind the scenes.

1) Prerequisites

Students should know:

- Basic PHP syntax (variables, arrays, functions, if/else)
- Basic HTML forms (GET/POST)
- Basic SQL (SELECT/INSERT idea)

Students will learn today:

- OOP in a real app (SRP, DI, encapsulation)
 - PDO + prepared statements + error handling
 - Secure form handling (CSRF, XSS escaping)
 - Front Controller routing (pretty URLs)
 - Clean architecture: controllers/router + views + model/repository
 - Patterns used by real frameworks: flash, old input, method spoofing, pagination
-

2) Laragon Setup (Very Important)

2.1 Create project folder

Create this folder:

```
C:\laragon\www\student-dashboard
```

Laragon will serve it automatically as:

```
http://student-dashboard.test
```

2.2 Start services

Laragon → **Start All**

2.3 Enable Pretty URLs (Apache)

Laragon usually uses Apache with `mod_rewrite` enabled.

If you get 404 routing issues:

- Laragon → Menu → Apache → Modules → ensure **rewrite_module** is enabled
 - Restart Apache
-

3) Database Setup

Run this SQL in HeidiSQL/phpMyAdmin:

```
CREATE DATABASE demo_oop;
USE demo_oop;

CREATE TABLE students (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP NULL DEFAULT NULL
);
```

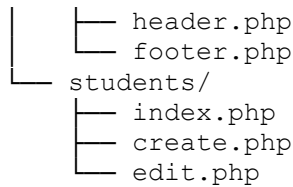
Insert sample rows:

```
INSERT INTO students (name, email) VALUES
('Alice Rahman', 'alice@example.com'),
('Bob Khan', 'bob@example.com'),
('Charlie Das', 'charlie@example.com');
```

✓ Advanced note: We added `updated_at` for real-world updates.

4) Project Structure (Advanced, Still Simple)

```
student-dashboard/
├── .htaccess
├── .env.example
├── config.php                (fallback config)
├── public/
│   ├── .htaccess
│   └── index.php             (Front Controller)
├── src/
│   ├── Core/
│   │   ├── App.php          (container-ish bootstrap)
│   │   ├── Router.php        (routing)
│   │   ├── Request.php       (request wrapper)
│   │   ├── Response.php      (response helpers)
│   │   ├── Session.php       (flash + old input)
│   │   ├── Csrf.php          (csrf)
│   │   ├── Validator.php     (rules)
│   │   └── Env.php           (.env loader)
│   ├── Database.php          (PDO connection + helpers)
│   └── Repositories/
│       └── StudentRepository.php (CRUD)
└── views/
    └── partials/
```



✓ This teaches real-world organization:

- **Core:** reusable infrastructure (router, request, session, csrf)
- **Repositories:** data access logic
- **Views:** UI only
- **public/index.php:** single entry point like frameworks

5) Routing (Pretty URLs)

URL	Method	Action
/	GET	Redirect to /students
/students	GET	List + search + pagination
/students/create	GET	Show create form
/students	POST	Store new student
/students/{id}/edit	GET	Show edit form
/students/{id}	POST + _method=PUT	Update student
/students/{id}	POST + _method=DELETE	Delete student

✓ Method spoofing is used because HTML forms support GET/POST only.

6) FINAL PROJECT CODE (COPY-PASTEABLE)

6.1 Root .htaccess (redirect to public/)

Create: student-dashboard/.htaccess

```
RewriteEngine On
```

```
RewriteCond %{REQUEST_URI} !^/public/  
RewriteRule ^(.*)$ public/$1 [L]
```

6.2 `public/.htaccess` (Front Controller routing)

Create: `student-dashboard/public/.htaccess`

```
RewriteEngine On

RewriteCond %{REQUEST_FILENAME} -f [OR]
RewriteCond %{REQUEST_FILENAME} -d
RewriteRule ^ - [L]

RewriteRule ^ index.php [QSA,L]
```

6.3 `.env.example` (real-world style)

Create: `student-dashboard/.env.example`

```
APP_ENV=local
APP_DEBUG=1

DB_HOST=localhost
DB_NAME=demo_oop
DB_USER=root
DB_PASS=
DB_CHARSET=utf8mb4
```

Students copy it to `.env` (optional) and edit credentials.

6.4 `config.php` fallback

Create: `student-dashboard/config.php`

```
<?php
declare(strict_types=1);

return [
    'app' => [
        'env' => 'local',
        'debug' => true,
    ],
    'db' => [
        'host' => 'localhost',
        'name' => 'demo_oop',
        'user' => 'root',
        'pass' => '',
        'charset' => 'utf8mb4',
    ],
];
```

6.5 `src/Core/Env.php` (tiny `.env` loader)

Create: `student-dashboard/src/Core/Env.php`

```
<?php
declare(strict_types=1);

namespace App\Core;

final class Env
{
    public static function load(string $path): void
    {
        if (!is_file($path)) {
            return; // .env is optional
        }

        $lines = file($path, FILE_IGNORE_NEW_LINES | FILE_SKIP_EMPTY_LINES);
        if ($lines === false) return;

        foreach ($lines as $line) {
            $line = trim($line);
            if ($line === '' || str_starts_with($line, '#')) continue;

            [$key, $value] = array_pad(explode('=', $line, 2), 2, '');
            $key = trim($key);
            $value = trim($value);

            // Remove optional quotes
            $value = trim($value, "\"'");

            if ($key !== '' && getenv($key) === false) {
                putenv($key . '=' . $value);
                $_ENV[$key] = $value;
            }
        }
    }

    public static function get(string $key, ?string $default = null): ?string
    {
        $val = $_ENV[$key] ?? getenv($key);
        if ($val === false || $val === null || $val === '') return $default;
        return (string)$val;
    }
}
```

6.6 `src/Core/Session.php` (flash + old input)

Create: `student-dashboard/src/Core/Session.php`

```

<?php
declare(strict_types=1);

namespace App\Core;

final class Session
{
    public static function start(): void
    {
        if (session_status() !== PHP_SESSION_ACTIVE) {
            session_start();
        }
    }

    public static function flash(string $key, string $value): void
    {
        self::start();
        $_SESSION['flash'][$key] = $value;
    }

    public static function getFlash(string $key): ?string
    {
        self::start();
        $val = $_SESSION['flash'][$key] ?? null;
        unset($_SESSION['flash'][$key]);
        return $val;
    }

    public static function setOld(array $data): void
    {
        self::start();
        $_SESSION['old'] = $data;
    }

    public static function old(string $key, string $default = ''): string
    {
        self::start();
        $old = $_SESSION['old'][$key] ?? $default;
        return is_string($old) ? $old : $default;
    }

    public static function clearOld(): void
    {
        self::start();
        unset($_SESSION['old']);
    }

    public static function setErrors(array $errors): void
    {
        self::start();
        $_SESSION['errors'] = $errors;
    }

    public static function errors(): array
    {
        self::start();
        $errors = $_SESSION['errors'] ?? [];
    }
}

```

```

        return is_array($errors) ? $errors : [];
    }

    public static function clearErrors(): void
    {
        self::start();
        unset($_SESSION['errors']);
    }
}

```

6.7 src/Core/Csrf.php

Create: student-dashboard/src/Core/Csrf.php

```

<?php
declare(strict_types=1);

namespace App\Core;

final class Csrf
{
    public static function token(): string
    {
        Session::start();
        if (empty($_SESSION['csrf'])) {
            $_SESSION['csrf'] = bin2hex(random_bytes(16));
        }
        return (string)$_SESSION['csrf'];
    }

    public static function verify(?string $token): bool
    {
        Session::start();
        $sessionToken = $_SESSION['csrf'] ?? '';
        return is_string($token) && is_string($sessionToken) && $sessionToken
            !== ''
                && hash_equals($sessionToken, $token);
    }
}

```

6.8 src/Core/Validator.php (rules + error bag)

Create: student-dashboard/src/Core/Validator.php

```

<?php
declare(strict_types=1);

namespace App\Core;

final class Validator
{

```



```

/** @return array<string, string> errors keyed by field */
public static function validateStudent(array $data): array
{
    $errors = [];

    $name = trim((string)($data['name'] ?? ''));
    $email = trim((string)($data['email'] ?? ''));

    if ($name === '') {
        $errors['name'] = 'Name is required.';
    } elseif (mb_strlen($name) < 3) {
        $errors['name'] = 'Name must be at least 3 characters.';
    } elseif (mb_strlen($name) > 100) {
        $errors['name'] = 'Name must be at most 100 characters.';
    }

    if ($email === '') {
        $errors['email'] = 'Email is required.';
    } elseif (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        $errors['email'] = 'Email format is invalid.';
    } elseif (mb_strlen($email) > 150) {
        $errors['email'] = 'Email must be at most 150 characters.';
    }

    return $errors;
}
}

```

6.9 src/Core/Request.php (HTTP wrapper + method spoofing)

Create: student-dashboard/src/Core/Request.php

```

<?php
declare(strict_types=1);

namespace App\Core;

final class Request
{
    public function method(): string
    {
        $method = strtoupper($_SERVER['REQUEST_METHOD'] ?? 'GET');

        // Method spoofing: POST + _method=PUT/DELETE (like Laravel)
        if ($method === 'POST') {
            $spoof = strtoupper((string)($_POST['_method'] ?? ''));
            if (in_array($spoof, ['PUT', 'DELETE'], true)) {
                return $spoof;
            }
        }

        return $method;
    }
}

```

```

public function path(): string
{
    $uri = $_SERVER['REQUEST_URI'] ?? '/';
    $path = parse_url($uri, PHP_URL_PATH) ?: '/';
    $path = rtrim($path, '/') ?: '/';
    return $path;
}

public function input(string $key, string $default = ''): string
{
    $value = $_POST[$key] ?? $_GET[$key] ?? $default;
    return is_string($value) ? $value : $default;
}

public function allPost(): array
{
    return is_array($_POST) ? $_POST : [];
}

public function query(string $key, string $default = ''): string
{
    $value = $_GET[$key] ?? $default;
    return is_string($value) ? $value : $default;
}
}

```

6.10 src/Core/Response.php (redirect + escaping)

Create: student-dashboard/src/Core/Response.php

```

<?php
declare(strict_types=1);

namespace App\Core;

final class Response
{
    public static function redirect(string $to): never
    {
        header("Location: {$to}");
        exit;
    }

    public static function e(string $value): string
    {
        return htmlspecialchars($value, ENT_QUOTES, 'UTF-8');
    }
}

```

6.11 src/Core/Router.php (tiny router with params)

Create: student-dashboard/src/Core/Router.php

```
<?php
declare(strict_types=1);

namespace App\Core;

final class Router
{
    /** @var array<int, array{method:string, pattern:string,
handler:callable}> */
    private array $routes = [];

    public function add(string $method, string $pattern, callable $handler):
void
    {
        $this->routes[] = [
            'method' => strtoupper($method),
            'pattern' => $pattern,
            'handler' => $handler,
        ];
    }

    public function dispatch(Request $req): void
    {
        $method = $req->method();
        $path = $req->path();

        foreach ($this->routes as $route) {
            if ($route['method'] !== $method) continue;

            $regex = $this->toRegex($route['pattern']);
            if (preg_match($regex, $path, $matches)) {
                // Extract named params only
                $params = array_filter($matches, fn($k) => !is_int($k),
ARRAY_FILTER_USE_KEY);
                ($route['handler'])($req, $params);
                return;
            }
        }

        http_response_code(404);
        echo "404 Not Found";
    }

    private function toRegex(string $pattern): string
    {
        // Convert /students/{id}/edit to regex with named group (?P<id>\d+)
        $regex = preg_replace('#\{(\w+)\}#', '(?P<$1>[^\s/]+)', $pattern);
        return '#^' . $regex . '$#';
    }
}
```

6.12 src/Database.php (PDO + helper queries)

Create: student-dashboard/src/Database.php

```
<?php
declare(strict_types=1);

namespace App;

use PDO;
use PDOException;

final class Database
{
    private PDO $pdo;

    public function __construct(array $cfg)
    {
        $dsn =
"mysql:host={$cfg['host']};dbname={$cfg['name']};charset={$cfg['charset']}";

        try {
            $this->pdo = new PDO(
                $dsn,
                $cfg['user'],
                $cfg['pass'],
                [
                    PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
                    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
                ]
            );
        } catch (PDOException $e) {
            // Production: log $e->getMessage()
            throw new PDOException("DB connection failed.");
        }
    }

    public function pdo(): PDO
    {
        return $this->pdo;
    }

    public function fetchAll(string $sql, array $params = []): array
    {
        $st = $this->pdo->prepare($sql);
        $st->execute($params);
        return $st->fetchAll();
    }

    public function fetch(string $sql, array $params = []): ?array
    {
        $st = $this->pdo->prepare($sql);
        $st->execute($params);
        $row = $st->fetch();
        return $row ?: null;
    }

    public function execute(string $sql, array $params = []): bool
```

```

    {
        $st = $this->pdo->prepare($sql);
        return $st->execute($params);
    }

    public function lastInsertId(): int
    {
        return (int)$this->pdo->lastInsertId();
    }
}

```

6.13 src/Repositories/StudentRepository.php (CRUD + pagination + search)

Create: student-dashboard/src/Repositories/StudentRepository.php

```

<?php
declare(strict_types=1);

namespace App\Repositories;

use App\Database;
use PDOException;

final class StudentRepository
{
    public function __construct(private Database $db) {}

    public function create(string $name, string $email): int
    {
        // Unique email check for friendly error
        $exists = $this->db->fetch("SELECT id FROM students WHERE email = :email", ['email' => $email]);
        if ($exists) {
            throw new \InvalidArgumentException("This email already exists.");
        }

        $ok = $this->db->execute(
            "INSERT INTO students (name, email) VALUES (:name, :email)",
            ['name' => $name, 'email' => $email]
        );

        if (!$ok) {
            throw new PDOException("Failed to create student.");
        }

        return $this->db->lastInsertId();
    }

    public function find(int $id): ?array
    {

```

```

        return $this->db->fetch("SELECT * FROM students WHERE id = :id",
['id' => $id]);
    }

    /** @return array{rows: array<int, array>, total: int, page: int,
perPage: int, pages: int} */
    public function paginate(int $page, int $perPage, string $searchEmail =
''): array
    {
        $page = max(1, $page);
        $perPage = max(1, min(50, $perPage));
        $offset = ($page - 1) * $perPage;

        $where = "";
        $params = [];

        if (trim($searchEmail) !== '') {
            $where = "WHERE email = :email";
            $params['email'] = trim($searchEmail);
        }

        $countRow = $this->db->fetch("SELECT COUNT(*) AS c FROM students
{$where}", $params);
        $total = (int)($countRow['c'] ?? 0);
        $pages = max(1, (int)ceil($total / $perPage));

        // Clamp page to last page
        $page = min($page, $pages);
        $offset = ($page - 1) * $perPage;

        $sql = "SELECT * FROM students {$where} ORDER BY id DESC LIMIT
{$perPage} OFFSET {$offset}";
        $rows = $this->db->fetchAll($sql, $params);

        return [
            'rows' => $rows,
            'total' => $total,
            'page' => $page,
            'perPage' => $perPage,
            'pages' => $pages,
        ];
    }

    public function update(int $id, string $name, string $email): bool
    {
        // Unique email check excluding current id
        $exists = $this->db->fetch(
            "SELECT id FROM students WHERE email = :email AND id != :id",
            ['email' => $email, 'id' => $id]
        );
        if ($exists) {
            throw new \InvalidArgumentException("This email already
exists.");
        }

        return $this->db->execute(

```

```

        "UPDATE students SET name = :name, email = :email, updated_at =
NOW() WHERE id = :id",
        ['id' => $id, 'name' => $name, 'email' => $email]
    );
}

public function delete(int $id): bool
{
    return $this->db->execute("DELETE FROM students WHERE id = :id",
['id' => $id]);
}
}

```

6.14 public/index.php (BOOTSTRAP + ROUTES)

Create: student-dashboard/public/index.php

```

<?php
declare(strict_types=1);

/**
 * Front Controller:
 * All requests come here via .htaccess rewriting.
 */

require_once __DIR__ . '/../src/Core/Env.php';
require_once __DIR__ . '/../src/Core/Session.php';
require_once __DIR__ . '/../src/Core/Csrf.php';
require_once __DIR__ . '/../src/Core/Validator.php';
require_once __DIR__ . '/../src/Core/Request.php';
require_once __DIR__ . '/../src/Core/Response.php';
require_once __DIR__ . '/../src/Core/Router.php';

require_once __DIR__ . '/../src/Database.php';
require_once __DIR__ . '/../src/Repositories/StudentRepository.php';

use App\Core\Env;
use App\Core\Session;
use App\Core\Csrf;
use App\Core\Validator;
use App\Core\Request;
use App\Core\Response;
use App\Core Router;

use App\Database;
use App\Repositories\StudentRepository;

// Load environment variables (.env optional)
Env::load(__DIR__ . '/../.env');

// Load fallback config
$config = require __DIR__ . '/../config.php';

// Merge env overrides if present (real-world style)

```

```

$appEnv = Env::get('APP_ENV', $config['app']['env']);
$appDebug = Env::get('APP_DEBUG', $config['app']['debug'] ? '1' : '0') ===
'1';

$dbCfg = [
    'host' => Env::get('DB_HOST', $config['db']['host']),
    'name' => Env::get('DB_NAME', $config['db']['name']),
    'user' => Env::get('DB_USER', $config['db']['user']),
    'pass' => Env::get('DB_PASS', $config['db']['pass']),
    'charset' => Env::get('DB_CHARSET', $config['db']['charset']),
];

// Start session
Session::start();

// Setup core services
$request = new Request();
$db = new Database($dbCfg);
$studentsRepo = new StudentRepository($db);

$router = new Router();

/**
 * -----
 * ROUTES (GET)
 * -----
 */
$router->add('GET', '/', function () {
    Response::redirect('/students');
});

$router->add('GET', '/students', function (Request $req) use ($studentsRepo)
{
    $success = Session::getFlash('success');
    $error = Session::getFlash('error');
    $errors = Session::errors();

    // Clear old/errors on fresh GET list
    Session::clearOld();
    Session::clearErrors();

    $search = trim($req->query('search', ''));
    $page = (int)$req->query('page', '1');
    $perPage = (int)$req->query('perPage', '8');

    $data = $studentsRepo->paginate($page, $perPage, $search);

    $students = $data['rows'];
    $meta = $data; // includes total/page/pages/perPage

    require __DIR__ . '/../views/students/index.php';
});

$router->add('GET', '/students/create', function () {
    $success = Session::getFlash('success');
    $error = Session::getFlash('error');
    $errors = Session::errors();

```



```

        require __DIR__ . '/../views/students/create.php';
    });

    $router->add('GET', '/students/{id}/edit', function (Request $req, array
$params) use ($studentsRepo) {
        $id = (int)($params['id'] ?? 0);
        $student = $studentsRepo->find($id);

        if (!$student) {
            Session::flash('error', 'Student not found. ');
            Response::redirect('/students');
        }

        $success = Session::getFlash('success');
        $error = Session::getFlash('error');
        $errors = Session::errors();

        require __DIR__ . '/../views/students/edit.php';
    });

/**
 * -----
 * ROUTES (POST/PUT/DELETE)
 * -----
 * Method spoofing supported by Request->method():
 * POST + _method=PUT or _method=DELETE
 */
$router->add('POST', '/students', function (Request $req) use ($studentsRepo)
{
    // Store
    $post = $req->allPost();

    if (!Csrf::verify($post['csrf'] ?? null)) {
        Session::flash('error', 'Invalid CSRF token. ');
        Response::redirect('/students');
    }

    $errors = Validator::validateStudent($post);
    if ($errors) {
        Session::setErrors($errors);
        Session::setOld($post);
        Response::redirect('/students/create');
    }

    $name = trim((string)$post['name']);
    $email = trim((string)$post['email']);

    try {
        $id = $studentsRepo->create($name, $email);
        Session::flash('success', "Student created successfully (ID:
{$id}). ");
        Response::redirect('/students');
    } catch (Throwable $e) {
        Session::flash('error', $e->getMessage());
        Session::setOld($post);
        Response::redirect('/students/create');
    }
}

```

```

});

$router->add('PUT', '/students/{id}', function (Request $req, array $params)
use ($studentsRepo) {
    $id = (int)($params['id'] ?? 0);
    $post = $req->allPost();

    if (!Csrf::verify($post['csrf'] ?? null)) {
        Session::flash('error', 'Invalid CSRF token.');
```

Response::redirect('/students');

```
    }

    $errors = Validator::validateStudent($post);
    if ($errors) {
        Session::setErrors($errors);
        Session::setOld($post);
        Response::redirect("/students/{id}/edit");
    }

    $name = trim((string)$post['name']);
    $email = trim((string)$post['email']);

    try {
        $studentsRepo->update($id, $name, $email);
        Session::flash('success', 'Student updated successfully.');
```

Response::redirect('/students');

```
    } catch (Throwable $e) {
        Session::flash('error', $e->getMessage());
        Session::setOld($post);
        Response::redirect("/students/{id}/edit");
    }
});

$router->add('DELETE', '/students/{id}', function (Request $req, array
$params) use ($studentsRepo) {
    $id = (int)($params['id'] ?? 0);
    $post = $req->allPost();

    if (!Csrf::verify($post['csrf'] ?? null)) {
        Session::flash('error', 'Invalid CSRF token.');
```

Response::redirect('/students');

```
    }

    $studentsRepo->delete($id);
    Session::flash('success', 'Student deleted successfully.');
```

Response::redirect('/students');

```
});

// Dispatch request
$router->dispatch($request);

```

6.15 Views (Bootstrap UI + errors + old input)

views/partials/header.php

Create: student-dashboard/views/partials/header.php

```
<?php
declare(strict_types=1);

use App\Core\Response;
?>
<!doctype html>
<html lang="en">
<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title><?= Response::e($title ?? 'Student Dashboard') ?></title>
    <link
href="https://cdn.jsdelivrivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css
" rel="stylesheet">
</head>
<body class="bg-light">
<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
    <div class="container">
        <a class="navbar-brand" href="/students">Student Dashboard</a>
        <div class="navbar-nav">
            <a class="nav-link" href="/students">Students</a>
            <a class="nav-link" href="/students/create">Add Student</a>
        </div>
    </div>
</nav>
<main class="container py-4">
```

views/partials/footer.php

Create: student-dashboard/views/partials/footer.php

```
</main>
<script
src="https://cdn.jsdelivrivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.mi
n.js"></script>
</body>
</html>
```

views/students/index.php (List + Search + Pagination + Delete)

Create: student-dashboard/views/students/index.php

```

<?php
declare(strict_types=1);

use App\Core\Response;
use App\Core\Csrf;

$title = 'Students';
require __DIR__ . '/../partials/header.php';

$search = (string)($_GET['search'] ?? '');
$page = (int)($meta['page'] ?? 1);
$pages = (int)($meta['pages'] ?? 1);
$perPage = (int)($meta['perPage'] ?? 8);
$total = (int)($meta['total'] ?? 0);
?>

<div class="d-flex justify-content-between align-items-center mb-3">
    <h1 class="h3 m-0">Students</h1>
    <a class="btn btn-primary" href="/students/create">+ Add Student</a>
</div>

<?php if (!empty($success)): ?>
    <div class="alert alert-success"><?= Response::e($success) ?></div>
<?php endif; ?>

<?php if (!empty($error)): ?>
    <div class="alert alert-danger"><?= Response::e($error) ?></div>
<?php endif; ?>

<div class="card shadow-sm mb-3">
    <div class="card-body">
        <form class="row g-2" method="get" action="/students">
            <div class="col-md-6">
                <input class="form-control" name="search" placeholder="Search by
exact email"
                    value="<?= Response::e($search) ?>">
            </div>

            <div class="col-md-3">
                <select class="form-select" name="perPage">
                    <?php foreach ([5,8,10,15,20] as $n): ?>
                        <option value="<?= $n ?>" <?= $n === $perPage ? 'selected' : ''
?><?= $n ?> per page</option>
                    <?php endforeach; ?>
                </select>
            </div>

            <div class="col-md-3 d-flex gap-2">
                <button class="btn btn-outline-primary w-100">Search</button>
                <a class="btn btn-outline-secondary w-100" href="/students">Reset</a>
            </div>
        </form>

        <div class="mt-2 small text-muted">
            Total: <strong><?= $total ?></strong> student(s)
        </div>
    </div>
</div>

```

```

</div>

<div class="card shadow-sm">
  <div class="card-body">
    <div class="table-responsive">
      <table class="table table-striped align-middle">
        <thead>
          <tr>
            <th style="width: 70px;">ID</th>
            <th>Name</th>
            <th>Email</th>
            <th style="width: 180px;">Created</th>
            <th style="width: 200px;" class="text-end">Actions</th>
          </tr>
        </thead>
        <tbody>
          <?php foreach ($students as $s): ?>
            <tr>
              <td><?= (int)$s['id'] ?></td>
              <td><?= Response::e($s['name']) ?></td>
              <td><?= Response::e($s['email']) ?></td>
              <td><?= Response::e($s['created_at']) ?></td>
              <td class="text-end">
                <a class="btn btn-sm btn-outline-secondary"
                  href="/students/<?= (int)$s['id'] ?>/edit">Edit</a>

                <form class="d-inline"
                  action="/students/<?= (int)$s['id'] ?>"
                  method="post"
                  onsubmit="return confirm('Delete this student?');">
                  <input type="hidden" name="csrf" value="<?=
Response::e(Csrf::token()) ?>">
                  <input type="hidden" name="_method" value="DELETE">
                  <button class="btn btn-sm btn-outline-danger">Delete</button>
                </form>
              </td>
            </tr>
          <?php endforeach; ?>
        </tbody>
      </table>

      <?php if (count($students) === 0): ?>
        <p class="text-muted mb-0">No results.</p>
      <?php endif; ?>
    </div>

    <!-- Pagination -->
    <?php if ($pages > 1): ?>
      <nav class="mt-3">
        <ul class="pagination mb-0">
          <?php
            $base = '/students?search=' . urlencode($search) . '&perPage=' .
$perPage . '&page=';
            ?>
          <li class="page-item <?= $page <= 1 ? 'disabled' : '' ?>">
            <a class="page-link" href="<?= $base . max(1, $page - 1)
?>">Prev</a>

```

```

</li>

<?php for ($p = 1; $p <= $pages; $p++): ?>
    <li class="page-item <?= $p === $page ? 'active' : '' ?>">
        <a class="page-link" href="<?= $base . $p ?>"><?= $p ?></a>
    </li>
<?php endfor; ?>

    <li class="page-item <?= $page >= $pages ? 'disabled' : '' ?>">
        <a class="page-link" href="<?= $base . min($pages, $page + 1)
?>">Next</a>
    </li>
</ul>
</nav>
<?php endif; ?>
</div>
</div>

<?php require __DIR__ . '/../partials/footer.php'; ?>

```

views/students/create.php (Create + field errors + old input)

Create: student-dashboard/views/students/create.php

```

<?php
declare(strict_types=1);

use App\Core\Response;
use App\Core\Csrf;
use App\Core\Session;

$title = 'Add Student';
require __DIR__ . '/../partials/header.php';

$errors = Session::errors();
$nameOld = Session::old('name');
$emailOld = Session::old('email');
?>

<h1 class="h3 mb-3">Add Student</h1>

<?php if (!empty($error)): ?>
    <div class="alert alert-danger"><?= Response::e($error) ?></div>
<?php endif; ?>

<div class="card shadow-sm">
    <div class="card-body">
        <form action="/students" method="post" class="row g-3">
            <input type="hidden" name="csrf" value="<?= Response::e(Csrf::token())
?>">

            <div class="col-md-6">
                <label class="form-label">Name</label>

```

```

        <input name="name" class="form-control <?= isset($errors['name']) ?
'is-invalid' : '' ?>"
            value="<?= Response::e($nameOld) ?>" placeholder="e.g., Alice
Rahman">
        <?php if (isset($errors['name'])): ?>
            <div class="invalid-feedback"><?= Response::e($errors['name'])
?></div>
        <?php endif; ?>
    </div>

    <div class="col-md-6">
        <label class="form-label">Email</label>
        <input name="email" type="email" class="form-control <?=
isset($errors['email']) ? 'is-invalid' : '' ?>"
            value="<?= Response::e($emailOld) ?>"
            placeholder="alice@example.com">
        <?php if (isset($errors['email'])): ?>
            <div class="invalid-feedback"><?= Response::e($errors['email'])
?></div>
        <?php endif; ?>
    </div>

    <div class="col-12 d-flex gap-2">
        <button class="btn btn-primary">Save</button>
        <a class="btn btn-outline-secondary" href="/students">Cancel</a>
    </div>
</form>
</div>
</div>

<?php require __DIR__ . '/../partials/footer.php'; ?>

```

views/students/edit.php (Edit + old input + PUT spoof)

Create: student-dashboard/views/students/edit.php

```

<?php
declare(strict_types=1);

use App\Core\Response;
use App\Core\Csrf;
use App\Core\Session;

$title = 'Edit Student';
require __DIR__ . '/../partials/header.php';

$errors = Session::errors();

// Use old input if validation failed, otherwise use DB values
$nameVal = Session::old('name', (string)$student['name']);
$emailVal = Session::old('email', (string)$student['email']);
?>

<h1 class="h3 mb-3">Edit Student</h1>

```

```

<?php if (!empty($error)): ?>
    <div class="alert alert-danger"><?= Response::e($error) ?></div>
<?php endif; ?>

<div class="card shadow-sm">
    <div class="card-body">
        <form action="/students/<?= (int)$student['id'] ?>" method="post"
class="row g-3">
            <input type="hidden" name="csrf" value="<?= Response::e(Csrf::token())
?>">
            <input type="hidden" name="_method" value="PUT">

            <div class="col-md-6">
                <label class="form-label">Name</label>
                <input name="name" class="form-control <?= isset($errors['name']) ?
'is-invalid' : '' ?>"
                    value="<?= Response::e($nameVal) ?>">
                <?php if (isset($errors['name'])): ?>
                    <div class="invalid-feedback"><?= Response::e($errors['name'])
?></div>
                <?php endif; ?>
            </div>

            <div class="col-md-6">
                <label class="form-label">Email</label>
                <input name="email" type="email" class="form-control <?=
isset($errors['email']) ? 'is-invalid' : '' ?>"
                    value="<?= Response::e($emailVal) ?>">
                <?php if (isset($errors['email'])): ?>
                    <div class="invalid-feedback"><?= Response::e($errors['email'])
?></div>
                <?php endif; ?>
            </div>

            <div class="col-12 d-flex gap-2">
                <button class="btn btn-primary">Update</button>
                <a class="btn btn-outline-secondary" href="/students">Back</a>
            </div>
        </form>
    </div>
</div>

<?php require __DIR__ . '/../partials/footer.php'; ?>

```

7) Advanced Explanations (For Your PDF)

7.1 Why Front Controller?

Front Controller means: one entry point (public/index.php) handles all requests. This is what Laravel does: public/index.php boots the framework and routes requests.

Benefits:

- Centralized security checks
- Clean URL routing
- Easier maintenance
- Real architecture

7.2 Why Router class?

Instead of `if/else` everywhere, we register routes:

- `GET /students`
- `POST /students`
- `PUT /students/{id}`
- `DELETE /students/{id}`

This is exactly how framework routing works.

7.3 Why Repository class?

Repository isolates **database logic**:

- Easier to read controllers
- Cleaner organization
- Swappable backend later (API, another DB)

7.4 Security Stack (must teach)

- **Prepared Statements** = stop SQL injection
- **CSRF Token** = stop cross-site request forgery
- **Escape Output** = stop XSS
- **Validation** = stop bad data & improve UX
- **Unique email handling** = real-world data integrity

7.5 PRG pattern (Post → Redirect → Get)

After POST, we redirect to GET.

Prevents:

- form resubmission
- duplicate insertions
- inconsistent UX

7.6 Old Input + Error Bag (Laravel-like UX)

When validation fails, we store:

- old input (so user doesn't retype)
 - errors (per-field messages)
-

8) Advanced “Bonus Upgrade” (Soft Delete) — Optional

If you want to add soft delete (advanced real-world feature):

Step 1: Alter table

```
ALTER TABLE students ADD deleted_at TIMESTAMP NULL DEFAULT NULL;
```

Step 2: Update repository queries

- For listing: add `WHERE deleted_at IS NULL`
- For delete: update `deleted_at = NOW()` instead of deleting row

This teaches real-world data safety.

9) Final Class Exercises (Advanced)

Beginner-to-Intermediate

1. Add `phone` column + validation rules
2. Add “View Student” page: `/students/{id}`
3. Add sorting: `?sort=name&dir=asc`

Intermediate

4. Add “search by name contains” (LIKE query safely)
5. Add “bulk delete” (checkbox selection + one POST)

Challenge (Portfolio Level)

6. Add Authentication:
 - `users` table

- password_hash
 - login + logout
 - protect routes (middleware concept)
-